

HƯỚNG DẪN TỪNG BƯỚC THIẾT LẬP VÀ VẬN HÀNH HỆ THỐNG QA AGENT

CO-WELL Tech Academy – Practical AI For Manual Testers

CO-WELL Tech Academy

Ngày 16 tháng 9 năm 2026

Mục lục

1	Giới Thiệu Chung & 4 Trụ Cột Triết Lý	3
2	Giai Đoạn 1: Khởi Tạo Môi Trường (Chỉ Có Thư Mục agents/)	4
2.1	Bước 1.1: Khởi tạo <code>package.json</code> và Cài đặt Thư viện	4
2.2	Bước 1.2: Thiết Lập Cấu Trúc Thư Mục Tiêu Chuẩn	4
2.3	Bước 1.3: Khởi Tạo Tri Thức Toàn Dự Án (<code>knowledge/</code>)	5
2.4	Bước 1.4: Đặt Hiến Pháp <code>AGENTS.md</code> Vào Thư Mục Gốc	5
3	Sơ Đồ Kiến Trúc & Mô Hình Điều Phối QA Leader	6
3.1	Sơ Đồ Vector TikZ: Luồng Điều Phối Một Cửa	6
3.2	Sơ Đồ Kiến Trúc Chi Tiết Đồ Họa	6
4	Quy Trình Vận Hành 100% Ngôn Ngữ Tự Nhiên (Zero-CLI)	7
4.1	Kịch Bản 1: Phân Tích Tính Năng Mới Từ File Của BA	7
4.2	Kịch Bản 2: Kiểm Tra Xung Đột Tri Thức Chéo (Cross-Feature Conflict)	8
4.3	Kịch Bản 3: Tự Động Dừng Lại Khi Gặp Kẽ Hở (Verdict Gate: ASK)	8
4.4	Kịch Bản 4: Sinh Test Cases Quy Mô Lớn (100 - 1.000 Test Cases)	9
4.5	Kịch Bản 5: Tiếp Tục Công Việc Sau Khi Mở Chat Mới (Zero-Path Resume)	9
5	Kết Nối Hệ Thống Quản Lý Kiểm Thử (Jira Xray & Redmine)	10
5.1	Chiến Lược Tích Hợp Tối Ưu (Hybrid Approach)	10
5.2	Kịch Bản 1: Xuất File CSV Chuẩn Cho Jira Xray	10
5.3	Kịch Bản 2: Xuất File CSV Chuẩn Cho Redmine	11
5.4	Kịch Bản 3: Dẩy Test Case Trực Tiếp Qua REST API	11
5.5	Kịch Bản 4: Kéo Danh Sách Lỗi Từ Jira Về Phân Tích Rủi Ro	12
6	Quy Trình Chỉnh Sửa Agent & Quản Lý Vùng Ảnh Hưởng (Agent Doctor)	13
6.1	Hiệu Ứng Domino Khi Chỉnh Sửa Agent	13
6.2	Kịch Bản 1: Sửa Đổi Bằng Ngôn Ngữ Tự Nhiên (Khuyến Dừng)	13
6.3	Kịch Bản 2: Kiểm Tra Toàn Vẹn Hệ Thống (Health Check)	14
6.4	Checklist 4 Bước Khi Thêm Hoặc Sửa Agent Mới	14
7	Tổng Kết: Bảng Tra Cứu Lệnh Tự Nhiên & Hướng Dẫn Build PDF	15
7.1	Bảng Tra Cứu Nhanh: Bạn Nói Gì → AI Tự Động Làm Gì?	15
7.2	Bảng Công Cụ Script Nội Bộ (Dành Cho AI Agent)	15
7.3	Hướng Dẫn Biên Dịch (Build) Thành File PDF	15

1 Giới Thiệu Chung & 4 Trụ Cột Triết Lý



Hình 1: CO-WELL Tech Academy – Practical AI For Manual Testers

Tài liệu này hướng dẫn chi tiết quy trình thiết lập dự án từ con số không: **khi bạn chỉ có sẵn thư mục agents/ và cài đặt môi trường Node.js / npm.**

Hệ thống được xây dựng trên 4 nguyên tắc kỹ thuật cốt tử:

1. **Mô hình 1 cửa qua QA Leader (Single Point of Contact):** Người dùng (Tester, BA, PO, Quản lý) chỉ cần giao tiếp bằng lời nói tự nhiên với QA Leader. Không cần nhớ tên các sub-agent con.
2. **Trải nghiệm Không Dòng Lệnh (Zero-CLI):** Người dùng **tuyệt đối không phải mở terminal để gõ lệnh npm**. Toàn bộ script kỹ thuật là công cụ chạy ngầm do AI Agent tự động kích hoạt.
3. **Knowledge là cốt lõi vĩnh viễn (SSOT):** Tri thức tích lũy từ tài liệu thô trong INPUT/, chốt các kẽ hở với BA/PO và lưu vĩnh viễn vào knowledge/. Càng dùng càng thông minh, không bao giờ hỏi lại điều đã giải quyết.
4. **Chống tràn token (Plan-First & Blueprint Batching):** Bắt buộc lập kế hoạch 00_plan.md trước khi làm và chia lô khi sinh số lượng lớn test cases (lên tới 1.000 test cases).

Trụ cột	Lợi ích thực tế cho dự án
Zero-CLI	Người dùng không cần kiến thức lập trình, chỉ cần ra lệnh tự nhiên như trò chuyện với đồng nghiệp.
System Map First	Cấm Agent đọc mò từng file, tra cứu bản đồ vệ tinh <code>_system_map.json</code> tiết kiệm 80% token.
Plan-First	Chia nhỏ milestone, không bị quá tải context window, chuyển đổi giữa các tool mượt mà.
Chuẩn FACT	100% bám tài liệu gốc, không tự bịa giả định, loại bỏ hoàn toàn hiện tượng ảo giác.

Bảng 1: Bảng tóm tắt giá trị của 4 trụ cột thiết kế

2 Giai Đoạn 1: Khởi Tạo Môi Trường (Chỉ Có Thư Mục `agents/`)

Giả sử bạn sao chép thư mục `agents/` sang một dự án mới hoàn toàn trống rỗng. Dưới đây là các bước thiết lập hạ tầng ban đầu.

2.1 Bước 1.1: Khởi tạo `package.json` và Cài đặt Thư viện

Mở terminal tại thư mục gốc dự án và chạy: Khởi tạo `package.json` và cài đặt thư viện đọc tài liệu:

```
# 1. Khởi tạo package.json
npm init -y

# 2. Cài đặt thư viện đọc file docx của BA
npm install mammoth
```

Thêm các script nội bộ dành riêng cho Agent vào file `package.json`:

```
"scripts": {
  "convert": "node agents/tools/convert.js",
  "knowledge:new": "node agents/tools/new-knowledge.js",
  "testcases:merge": "node agents/tools/merge-testcases.js",
  "status": "node agents/tools/status.js",
  "map:sync": "node agents/tools/sync-system-map.js"
}
```

2.2 Bước 1.2: Thiết Lập Cấu Trúc Thư Mục Tiêu Chuẩn

Tạo các thư mục chức năng cơ bản theo cấu trúc:

```
mkdir INPUT OUTPUT knowledge knowledge/features
```

Sau bước này, cây thư mục tối giản ngoài root sẽ như sau:

```
<Project-Root>/
|-- INPUT/          # Nơi tha tài liệu yêu cầu của BA (.docx, .md)
|-- OUTPUT/         # Nơi xem sản phẩm phân tích & testcase
|-- knowledge/      # Nơi lưu trữ thực tiễn viên (SSOT)
|   '-- features/   # Thực tiễn tình hình
'-- agents/         # Hệ thống chuyên gia QA và công cụ nội bộ
```

2.3 Bước 1.3: Khởi Tạo Tri Thức Toàn Dự Án (knowledge/)

Tạo 4 file nền tảng trong thư mục `knowledge/`:

- `_project.md`: Điền quy ước cơ bản (tiền tệ: VNĐ, múi giờ: GMT+7, format mã định danh, chính sách NULL/rỗng).
- `_glossary.md`: Từ điển thuật ngữ nghiệp vụ cốt lõi của dự án (tránh nhầm lẫn giữa BA và Tester).
- `_template.md`: Bản mẫu chuẩn 10 mục dùng để nhân bản cho từng tính năng.
- `_system_map.json`: Bản đồ vệ tinh toàn hệ thống định tuyến các file.

Sau đó chạy lệnh đồng bộ bản đồ lần đầu:

```
npm run map:sync
```

2.4 Bước 1.4: Đặt Hiến Pháp AGENTS.md Vào Thư Mục Gốc

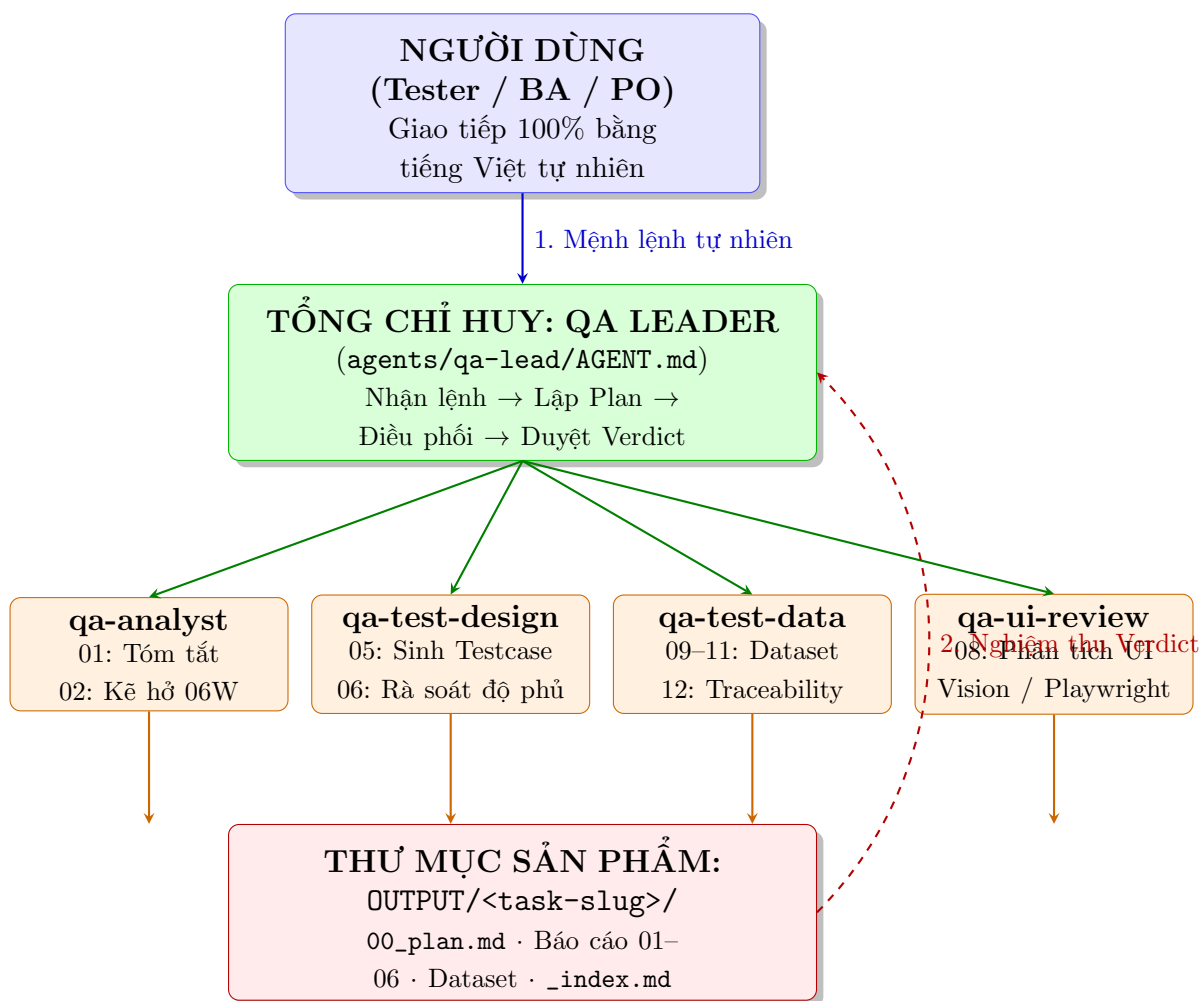
Đặt file `AGENTS.md` tại thư mục gốc. Đây là file chỉ huy cao nhất giúp mọi công cụ AI (**Antigravity IDE, Claude Code, Cursor, Codex, Gemini**) tự động nạp luật và tuân thủ nguyên tắc FACT, không cần nhắc lại.

3 Sơ Đồ Kiến Trúc & Mô Hình Điều Phối QA Leader

Chương này minh họa trực quan cấu trúc điều phối của hệ sinh thái Agent bằng sơ đồ vector **TikZ** sắc nét chuẩn xuất bản và hướng dẫn cách nhúng ảnh bên ngoài.

3.1 Sơ Đồ Vector TikZ: Luồng Điều Phối Một Cửa

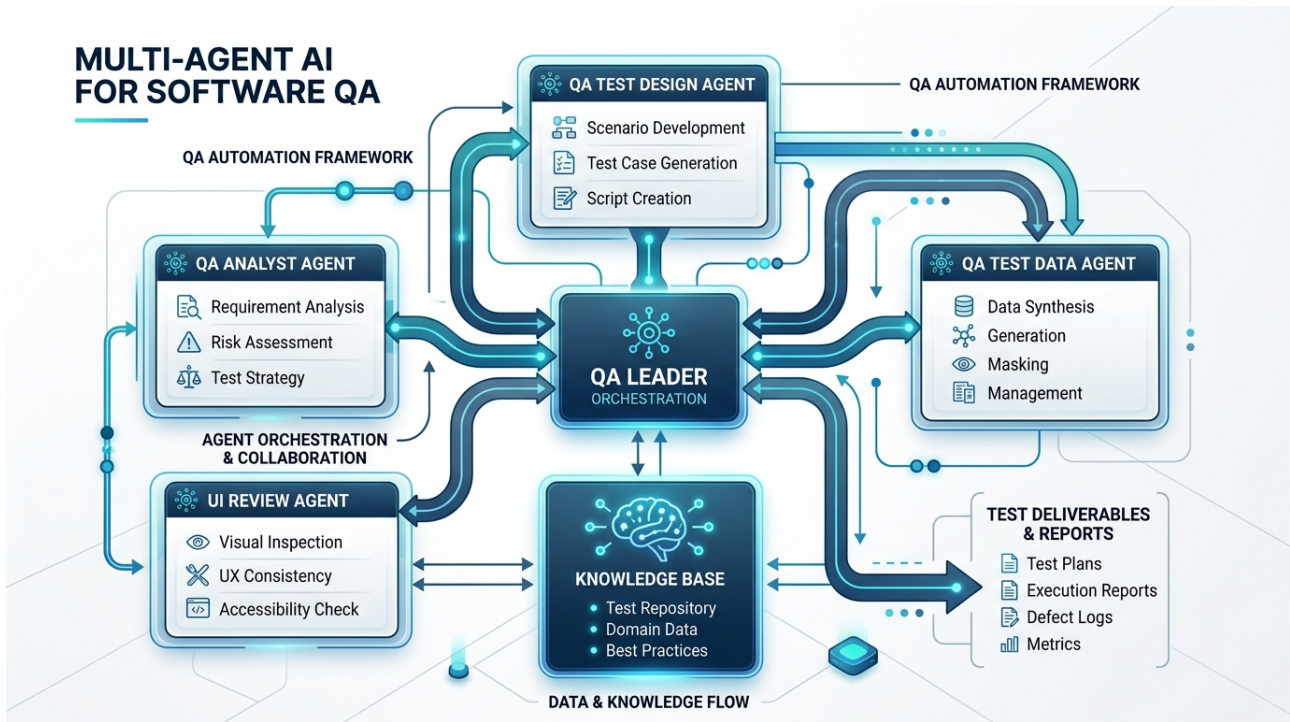
Sơ đồ dưới đây được vẽ trực tiếp bằng mã nguồn TikZ, tự động co giãn và sắc nét ở mọi độ phân giải mà không cần file ảnh nhúng kèm:



Hình 2: Kiến trúc điều phối phân tầng tập trung qua QA Leader

3.2 Sơ Đồ Kiến Trúc Chi Tiết Đồ Họa

Ngoài sơ đồ vector TikZ ở trên, hình 3 minh họa cấu trúc chi tiết của trung tâm điều phối QA Leader, bộ não tri thức Knowledge Base và các luồng xuất bản phẩm:



Hình 3: Sơ đồ tổng quan kiến trúc hệ thống Multi-Agent AI for Software QA

4 Quy Trình Vận Hành 100% Ngôn Ngữ Tự Nhiên (Zero-CLI)

Từ thời điểm hệ thống được khởi tạo, **người dùng (Tester, BA, PO, Quản lý)** không cần gõ bất kỳ câu lệnh terminal nào. Toàn bộ tương tác đều diễn ra dưới hình thức hội thoại tự nhiên với QA Leader.

4.1 Kịch Bản 1: Phân Tích Tính Năng Mới Từ File Của BA

1. **Thao tác người dùng:** Thả file tài liệu yêu cầu của BA vào thư mục INPUT/ (ví dụ: INPUT/Function_D.docx hoặc .md).

2. **Người dùng ra lệnh:**

"QA Lead ơi, tôi vừa bỏ tài liệu Function D vào INPUT, hãy tạo kế hoạch và phân tích tính năng này giúp tôi."

3. **Hành động tự động của Agent:**

- Tự chạy ngầm công cụ chuyển đổi `convert.js` nếu là file `.docx`.
- Tự khởi tạo hồ sơ tri thức tính năng `knowledge/features/function-d.md` từ mẫu chuẩn 10 phần.
- Thiết lập ngay kế hoạch kiểm soát ngữ cảnh `OUTPUT/function-d/00_plan.md`.
- Điều phối `qa-analyst` bóc tách 10 phần yêu cầu và chạy ma trận 06W tìm kẻ hở logic.

4.2 Kịch Bản 2: Kiểm Tra Xung Đột Tri Thức Chéo (Cross-Feature Conflict)

1. **Bối cảnh:** Khi một dự án có nhiều tính năng (Voucher, Giảm hàng, Flash Sale, Đăng ký thành viên), các quy tắc rất dễ đá nhau (ví dụ: Tính năng A bắt đăng nhập, Tính năng B cho khách vắng lai mua hàng; hoặc chính sách cộng dồn khuyến mãi bị mâu thuẫn).

2. **Người dùng ra lệnh:**

"Kiểm tra xem tính năng mới này có đá logic với các tính năng cũ trong dự án không?"

3. **Hành động tự động của Agent:**

- QA Leader kích hoạt công cụ ngầm `conflict-detector.js`.
- Hệ thống tự động đọc toàn bộ các file trong `knowledge/features/*.md`, đối chiếu chéo qua 4 nhóm quy tắc nhạy cảm:
 - (a) **Điều kiện tài khoản:** Khách vắng lai (Guest) vs Bắt buộc đăng nhập.
 - (b) **Cộng dồn khuyến mãi:** Cho phép vs Cấm áp dụng đồng thời.
 - (c) **Phạm vi giảm trừ:** Tiền hàng vs Phí vận chuyển.
 - (d) **Chính sách hoàn huỷ:** Hoàn trả quyền lợi vs Hủy đơn mất luôn.
- Xuất báo cáo xung đột vào `OUTPUT/<slug>/01_conflict_warning.md`.
- Nếu có mâu thuẫn, Agent cảnh báo ngay và ra verdict **ASK** để người dùng chốt với BA.

4.3 Kịch Bản 3: Tự Động Dừng Lại Khi Gặp Kẽ Hở (Verdict Gate: ASK)

1. Khi quét 06W phát hiện kẽ hở logic mà tài liệu chưa nêu rõ (ví dụ: quy tắc làm tròn tiền lẻ của mã % hoặc xử lý khi rút mạng):

- QA Leader **dừng ngay pipeline**, nghiêm cấm tự ý suy đoán bừa bãi.
- Trình bày câu hỏi rõ ràng ra khung chat:

"Hệ thống phát hiện kẽ hở [MR-06]: Tài liệu chưa nêu quy tắc làm tròn tiền lẻ cho voucher %. Tôi đề xuất làm tròn xuống hàng đồng VND. Bạn vui lòng xác nhận với BA nhé."

2. **Người dùng phản hồi:**

"BA đã chốt: Đồng ý làm tròn xuống hàng đồng và mã không phân biệt hoa thường nhé."

3. **Agent tự động ghi nhận vĩnh viễn:**

- Nạp câu trả lời vào Mục 8 (GIẢ ĐỊNH ĐÃ CHỐT) trong `knowledge/features/function-d.md`.
- Chuyển rule sang trạng thái **Confirmed** và tự động đồng bộ vệ tinh `_system_map.json`.
- Kể từ nay về sau, mọi Agent khi chạy đều kế thừa tri thức này mà không bao giờ hỏi lại.

4.4 Kịch Bản 4: Sinh Test Cases Quy Mô Lớn (100 - 1.000 Test Cases)

Để tránh tràn cửa sổ ngữ cảnh (Context Window) khi sinh hàng trăm test case, hệ thống áp dụng quy trình chia lô:

1. **Tạo Blueprint JSON:** Sinh danh mục rút gọn nhẹ (30 token/item) vào `05_test_blueprint.json`.
2. **Sinh chi tiết theo lô:** Giao qa-test-design viết đầy đủ 8 trường chuẩn cho từng lô 50 test cases vào `testcases/batch_01.md`, `batch_02.md`...
3. **Người dùng ra lệnh gộp:**
"Gộp các test case lại giúp tôi."
4. Agent tự động chạy công cụ ghép các lô thành file đặc tả hoàn chỉnh `05_test_case_spec.md`.

4.5 Kịch Bản 5: Tiếp Tục Công Việc Sau Khi Mở Chat Mới (Zero-Path Resume)

Nếu đang làm việc mà người dùng tắt máy, ngắt mạng hoặc chuyển sang ngày làm việc mới:

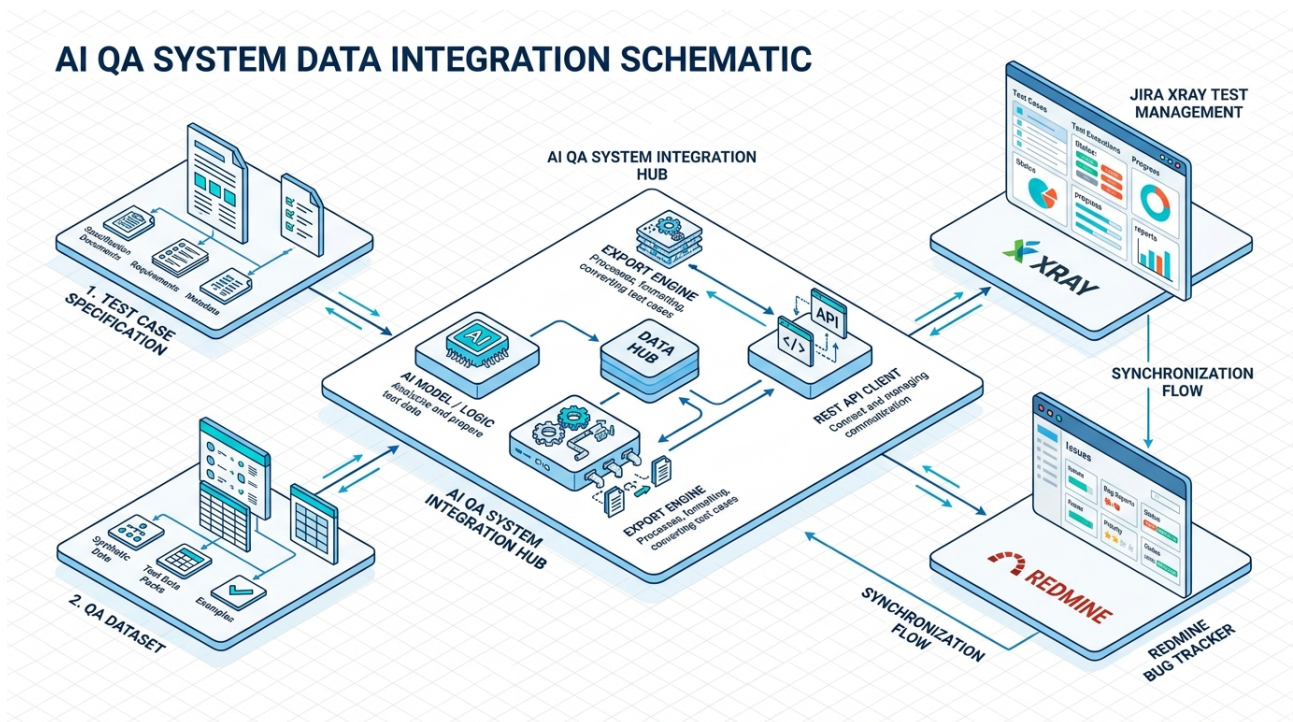
1. Người dùng chỉ cần mở khung chat và gõ câu ngắn gọn:
"Tiếp tục" hoặc "Tiến độ thế nào rồi?"
2. QA Leader tự động đọc `00_plan.md` tại thư mục `OUTPUT/`, kiểm tra các chặng đã xong và báo cáo:
"Tính năng Function D đã hoàn thành Chặng 1, 2, 3 và 4. Đang sẵn sàng để sinh Test Cases (Chặng 5). Bạn có muốn tôi tiến hành không?"
3. Người dùng gõ: *"Tiến hành"* → Agent tiếp tục ngay từ Chặng 5 mà không làm lại bước cũ, tiết kiệm 100% token đọc lại.

5 Kết Nối Hệ Thống Quản Lý Kiểm Thử (Jira Xray & Redmine)

Để phục vụ quản lý dự án thực tế, hệ thống hỗ trợ tích hợp hai chiều với **Jira Xray** và **Redmine** thông qua cả 2 hình thức: **Xuất file CSV** chuẩn hóa và **REST API Client** tự động.

5.1 Chiến Lược Tích Hợp Tối Ưu (Hybrid Approach)

- **Đẩy hàng loạt Test Cases (50 - 500 test cases):** Sử dụng xuất file CSV chuẩn (mã hóa UTF-8 BOM) hoặc REST API Client chạy ngầm. Tránh gọi MCP tool hàng trăm lần gây tràn context window.
- **Kéo lỗi (Defects/Bugs) từ Jira về:** Sử dụng REST API Client kéo danh sách bug của module về lưu vào file `jira_defects_summary.md` để qa-analyst đọc và bổ sung vào vùng rủi ro.



Hình 4: Sơ đồ luồng dữ liệu tích hợp hai chiều với Jira Xray và Redmine

5.2 Kịch Bản 1: Xuất File CSV Chuẩn Cho Jira Xray

1. Người dùng yêu cầu:

"Xuất test case ra file cho Jira Xray."

2. Xử lý của Agent:

- QA Leader tự động kích hoạt `export-testcases.js`.
- Đọc file đặc tả `05_test_case_spec.md`, bóc tách từng test case thành 10 cột chuẩn của Xray Test Importer:

Issue Type, Issue Key, Summary, Preconditions, Action, Data, Expected Result, Priority, Labels, Component

- File được lưu tại: OUTPUT/<task-slug>/export_jira_xray.csv.
- **Đặc tính UTF-8 BOM:** File được gắn Byte Order Mark (\uFEFF), người dùng có thể nhấp đúp mở trực tiếp bằng Microsoft Excel mà không bị lỗi dấu tiếng Việt.

3. Cách import vào Jira:

- (a) Trên Jira, truy cập dự án → Chọn **Xray Settings** → **Test Case Importer**.
- (b) Chọn file `export_jira_xray.csv`.
- (c) Bấm **Import** → Toàn bộ Test Cases được tạo tự động với đầy đủ Preconditions và các bước Manual Steps.

5.3 Kịch Bản 2: Xuất File CSV Chuẩn Cho Redmine

1. Người dùng yêu cầu:

"Xuất test case ra file cho Redmine."

2. Xử lý của Agent:

- Hệ thống sinh file: OUTPUT/<task-slug>/export_redmine.csv.
- Bao gồm 6 cột chuẩn: **Tracker** (*Test Case*), **Subject**, **Description** (chứa toàn bộ mục tiêu, tiền điều kiện, các bước, data, kỳ vọng dưới định dạng Textile/Markdown), **Priority**, **Category**, **Status** (*New*).

3. Cách import vào Redmine:

- (a) Vào dự án trên Redmine → Menu **Issues** → Nhấp **Import** (ở góc phải phía dưới).
- (b) Chọn file `export_redmine.csv`, cấu hình dấu phân cách là dấu phẩy (,) và mã hóa UTF-8.
- (c) Bấm **Import** để hoàn tất.

5.4 Kịch Bản 3: Đẩy Test Case Trực Tiếp Qua REST API

Nếu đội ngũ muốn đẩy tự động qua API thay vì tải file:

1. Thiết lập cấu hình trong file `.env` tại thư mục gốc:

```
JIRA_HOST=https://your-domain.atlassian.net
JIRA_EMAIL=qa-lead@shopgo.vn
JIRA_API_TOKEN=your_jira_api_token
JIRA_PROJECT_KEY=SHOPGO
```

2. Người dùng chỉ cần nói:

"Đẩy test case lên Jira giúp tôi."

3. Agent chạy ngầm `jira-client.js push <slug>`, kết nối REST API để tạo issue hàng loạt.

5.5 Kịch Bản 4: Kéo Danh Sách Lỗi Từ Jira Về Phân Tích Rủi Ro

1. Người dùng yêu cầu:

"Kéo các bug liên quan từ Jira về để phân tích."

2. Agent thực hiện:

- Chạy ngầm `jira-client.js pull <slug>`.
- Truy vấn qua JQL: `project = "SHOPGO"AND issuetype in (Bug, Defect) ORDER BY created DESC`.
- Tạo báo cáo: `OUTPUT/<task-slug>/jira_defects_summary.md`.
- `qa-analyst` đọc báo cáo này để nhận diện các vùng thường xuyên phát sinh lỗi (Defect-prone areas) và bổ sung thêm Viewpoint kiểm thử tương ứng.

6 Quy Trình Chỉnh Sửa Agent & Quản Lý Vùng Ảnh Hưởng (Agent Doctor)

Trong một hệ thống Multi-Agent chuyên nghiệp, các Agent không hoạt động độc lập mà liên kết chặt chẽ thông qua các **Hợp đồng Đầu vào / Đầu ra (Input/Output Contracts)**.

6.1 Hiệu Ứng Domino Khi Chỉnh Sửa Agent

Ví dụ: Nếu bạn muốn sửa đổi Agent `qa-analyst` để đổi định dạng mã Business Rule từ `BR-xx` sang `REQ-xx`:

- `qa-analyst` (Bước 01) sinh ra mã `BR-xx`.
- Các thành phần phụ thuộc phía sau bị gãy:
 1. Bước 02 (`02-missing-rule-06w`): Đọc `BR-xx` để tìm kẻ hở → **Lỗi**.
 2. Bước 04 (`04-test-idea-design`): Gắn nhãn `BR-xx` vào ý tưởng test → **Lỗi**.
 3. Bước 05 (`05-test-case-generation`): Gắn `Rule#BR-xx` vào trường Tags của Test Case → **Lỗi**.
 4. Bước 06 (`06-coverage-review`): Rà soát độ phủ 100% các mã `BR-xx` → **Báo thiếu độ phủ (0%)**.
 5. Bước 09 (`09-data-class-map`): Ánh xạ Data Class vào `BR-xx` → **Lỗi**.

Nếu chỉnh sửa thủ công mò mẫm, bạn rất dễ bỏ sót các liên kết ngầm này. Do đó, hệ thống trang bị công cụ **Agent Doctor** (`agents/tools/agent-doctor.js`) để bảo đảm an toàn tuyệt đối.

6.2 Kịch Bản 1: Sửa Đổi Bằng Ngôn Ngữ Tự Nhiên (Khuyến Dùng)

Người dùng không cần tìm kiếm các file code, chỉ cần trao đổi tự nhiên trong khung chat:

1. Người dùng yêu cầu:

"Tôi muốn sửa agent `qa-analyst`: đổi định dạng Business Rule từ `BR-xx` thành `RULE-xx`. Hãy phân tích các thành phần bị ảnh hưởng và sửa đồng bộ giúp tôi."

2. Quy trình Agent tự động thực hiện:

- (a) Tự động chạy `agent-doctor.js -impact qa-analyst`.
- (b) Báo cáo lại cho bạn bằng danh sách các bước bị ảnh hưởng (Bước 02, 03, 04, 05, 06, 09).
- (c) Tiến hành chỉnh sửa file gốc tại `agents/qa-analyst/skills/01-requirement-risk-summary.js`.
- (d) Tự động cập nhật đồng bộ định dạng tại tất cả các skill tiêu thụ phía sau.
- (e) Cập nhật lại `agents/qa-lead/AGENT.md` và chạy đồng bộ bản đồ `_system_map.json`.
- (f) Chạy lệnh kiểm tra `agent:check` để xác nhận 100% hệ thống đã đồng bộ.

6.3 Kịch Bản 2: Kiểm Tra Toàn Vẹn Hệ Thống (Health Check)

Bất kỳ lúc nào bạn muốn kiểm tra xem hệ thống có bị gãy liên kết hay mất file nào không, bạn chỉ cần yêu cầu:

"Kiểm tra sức khỏe toàn bộ hệ thống Agent giúp tôi."

Agent sẽ kích hoạt kiểm tra nhanh (`npm run agent:check`) và in ra bảng báo cáo trực quan:

Tên Agent	Vai trò chính	Trạng thái
qa-lead	Tổng chỉ huy & Điều phối Zero-CLI	Đạt chuẩn
qa-analyst	Phân tích 01 → 04 & Quét xung đột	Đạt chuẩn
qa-test-design	Sinh Test Case 05, Rà soát 06 & Xuất CSV	Đạt chuẩn
qa-test-data	Dataset 09 → 12 & Traceability Data	Đạt chuẩn
qa-exploratory	Kiểm thử thăm dò theo Charter 07	Đạt chuẩn
qa-ui-review	So sánh giao diện & Wireframe 08	Đạt chuẩn

6.4 Checklist 4 Bước Khi Thêm Hoặc Sửa Agent Mới

Khi đội ngũ phát triển muốn bổ sung một Agent chuyên biệt mới vào hệ sinh thái:

- Bước 1:** Tạo thư mục `agents/<ten-agent>/AGENT.md` mô tả quyền hạn, đầu vào, đầu ra và chuẩn FACT.
- Bước 2:** Đặt các quy trình thực thi vào thư mục con `agents/<ten-agent>/skills/`.
- Bước 3:** Đăng ký Agent mới vào bảng điều phối tại `agents/qa-lead/AGENT.md`.
- Bước 4:** Chạy đồng bộ bản đồ vệ tinh để mọi Agent khác tự động nhận biết vị trí của Agent mới:

`npm run map:sync`

7 Tổng Kết: Bảng Tra Cứu Lệnh Tự Nhiên & Hướng Dẫn Build PDF

7.1 Bảng Tra Cứu Nhanh: Bạn Nói Gì → AI Tự Động Làm Gì?

Câu Lệnh Tự Nhiên Của Bạn	Hành Động Tự Động Của QA Leader Ở Hậu Trường
"Tôi vừa bỏ file PRD vào INPUT, phân tích giúp tôi"	Tự convert .docx → .md, tạo 00_plan.md, giao qa-analyst bóc tách 10 phần và quét 06W.
"Kiểm tra xem tính năng mới có đá logic với tính năng cũ không"	Chạy ngầm conflict-detector.js, đối chiếu các file trong knowledge/features/, cảnh báo nếu có mâu thuẫn rule.
"Tiến độ thế nào rồi?"/ "Làm tiếp đi"	Đọc 00_plan.md tại OUTPUT/, báo cáo dashboard tiến độ và tiếp tục đúng chặng dang dở (Zero-Path Resume).
"BA đã chốt: [nội dung giải đáp]"	Nạp ngay vào Mục 8 của file tri thức tính năng và tự đồng bộ _system_map.json vĩnh viễn.
"Gộp các test case lại để tôi nghiệm thu"	Chạy ngầm merge-testcases.js ghép các batch thành 05_test_case_spec.md.
"Xuất file cho Jira Xray và Redmine"	Chạy ngầm export-testcases.js, tạo export_jira_xray.csv và export_redmine.csv có gán UTF-8 BOM chuẩn.
"Đẩy test case lên Jira"	Chạy ngầm jira-client.js push kết nối REST API để tạo issue hàng loạt.
"Kéo các bug từ Jira về để phân tích"	Chạy ngầm jira-client.js pull kéo danh sách defect về lưu vào jira_defects_summary.md.
"Tôi muốn sửa agent X... hãy kiểm tra tác động"	Kích hoạt agent-doctor.js -impact, rà soát vùng ảnh hưởng (Blast Radius) và sửa đồng bộ các bước liên quan.

Bảng 2: Bảng ánh xạ câu lệnh tự nhiên sang hành vi của hệ thống

7.2 Bảng Công Cụ Script Nội Bộ (Dành Cho AI Agent)

7.3 Hướng Dẫn Biên Dịch (Build) Thành File PDF

Bạn có thể tự biên dịch bộ tài liệu này thành file PDF hoàn chỉnh qua 3 cách đơn giản sau:

Cách 1: Biên dịch Online qua Overleaf (Dễ nhất - Không cần cài đặt):

- Nén thư mục .docs/latex/ thành file .zip.
- Truy cập <https://www.overleaf.com> → Đăng nhập → Chọn **New Project** → **Upload Project**.
- Bấm nút **Recompile** (Chọn Compiler: **XeLaTeX** để hỗ trợ tiếng Việt Unicode tốt nhất) → Tải file PDF về máy.

Cách 2: Biên dịch trong VS Code / Antigravity IDE:

Lệnh npm	Công cụ phụ trách	Mục đích sử dụng
npm run convert	convert.js	Chuyển đổi tài liệu .docx sang .md
npm run knowledge:new	new-knowledge.js	Tạo file tri thức mới theo khung chuẩn 10 phần
npm run testcases:merge	merge-testcases.js	Gộp các batch test cases thành file spec tổng
npm run testcases:export	export-testcases.js	Xuất test case sang CSV chuẩn Jira Xray & Redmine
npm run jira:push	jira-client.js push	Đẩy bulk test cases lên Jira qua REST API
npm run jira:pull	jira-client.js pull	Kéo danh sách open defects từ Jira qua JQL
npm run knowledge:conflicts	conflict-detector.js	Quét phát hiện xung đột chéo giữa các tính năng
npm run status	status.js	Báo cáo Dashboard tiến độ các task trong OUTPUT/
npm run map:sync	sync-system-map.js	Đồng bộ bản đồ vệ tinh _system_map.json
npm run agent:check	agent-doctor.js	Kiểm tra sức khỏe và phân tích tác động khi sửa Agent

Bảng 3: Danh mục công cụ nội bộ của QA Leader

- Cài extension **LaTeX Workshop**.
- Mở file .docs/latex/main.tex → Nhấn tổ hợp phím Ctrl + Alt + B để tự động build PDF.

Cách 3: Biên dịch bằng dòng lệnh:

```
cd .docs/latex
xelatex main.tex
# Or: pdflatex main.tex
```

Cách 4: Xem & In Trực Tiếp Bằng Trình Duyệt Web (Tức thì):

- Mở file .docs/preview.html bằng bất kỳ trình duyệt nào (Chrome, Edge, Firefox).
- Nhấn Ctrl + P → Chọn **Save as PDF** → Nhận ngay tài liệu định dạng đẹp chuẩn in ấn.